

Full-Stack MERN Assessment: Attendance Management System

Overview

Build a full-stack Attendance Management System that tracks employee attendance using location and live photo verification. The system should support role-based access and basic workflow management.

Tech Stack Requirements

- Frontend: React.js (Vite preferred)
 - State Management: Redux Toolkit + RTK Query
 - Backend: Node.js + Express.js
 - Database: MongoDB
 - Logging: Winston or Morgan (backend)
-

User Roles

- Employee
 - Manager
 - Admin
-

Core Requirements (MUST HAVE)

1. Authentication & Authorization

- Secure login and signup
 - Role-based access control (RBAC)
 - Protected routes (frontend + backend)
-

2. Attendance (Punch In / Punch Out)

- Employee can:
 - Punch In and Punch Out
 - Capture **live selfie using camera** (no file upload)
 - Capture **location (latitude & longitude)**
- Store:
 - Punch-in time
 - Punch-out time

- Selfie (image)
 - Location
 - Total working hours
-

3. Working Hours Logic

- Standard shift: 8 hours

Display:

- Completed $\rightarrow \geq 8$ hours
 - Incomplete $\rightarrow < 8$ hours
-

4. Overtime (OT) Workflow

- Employee can request overtime
 - Manager/Admin can:
 - Approve or Reject
 - Status must reflect in attendance
-

5. Dashboards

Employee

- View personal attendance
- View working hours
- Track overtime requests

Manager

- View team attendance
- View pending overtime requests

Admin

- View all users
 - Monitor system-wide attendance
-

6. Attendance Validation (IMPORTANT)

- Admin/Manager must be able to:
 - View employee selfies

- Verify authenticity
 - Mark attendance as:
 - Valid
 - Invalid (fake/suspicious)
 - Add remarks/notes
-

7. Reports

- Generate daily attendance report

Include:

- Name
- Punch-in/out time
- Selfie
- Location
- Working hours
- Status

Access:

- Employee → Own data
 - Manager → Team data
 - Admin → All data
-

Good to Have (Not Mandatory)

- Filters (date range, user)
 - Pagination
 - Clean UI/UX
-

Bonus Features (Optional)

- Geofencing (restrict attendance within radius)
- Notification system:
 - Missed punch alerts
 - Overtime approval/rejection alerts
- Real-time updates (Socket.IO)

- Dark mode
 - Export reports (PDF/Excel)
-

Deployment

- Frontend: Vercel / Netlify
 - Backend: Render / similar
 - Must be functional
-

Submission Requirements

- Public GitHub repository
 - Live deployed links (Frontend + Backend)
 - README.md
-

README Must Include

- Setup instructions
 - Architecture overview
 - Features implemented
 - Assumptions made
-

Evaluation Criteria

1. Core Feature Implementation (40%)

- Attendance flow
- RBAC
- Overtime workflow

2. Code Quality (20%)

- Clean structure
- Error handling
- Validation

3. UI & UX (15%)

- Usability
- Clarity

4. Architecture (15%)

- Scalability
- Separation of concerns

5. Bonus Features (10%)

Timeline

48 Hours

Note

Focus on correctness and clean implementation over completing all features. Partial but well-structured solutions are preferred over incomplete complex implementations.